One model.
One process.
OpenAI-ready.

Chat, completions and embeddings from one GGUF or safetensors model, with streaming, continuous batching, tool calls and structured output.

THE PROBLEM

Serving one model should be one command.

- **Glue code**

  Wrapping an engine in an OpenAI-shaped API is work every app repeats.

- **Queued requests**

  One request at a time wastes the hardware you already have.

- **Format friction**

  A Hugging Face model directory isn't a GGUF file, and conversion drags in torch.

WHAT SHIPS

Fast where it counts.

Continuous batching

--parallel N slots share one batch per tick, so requests generate together.

Structured output

GBNF grammars, json_schema and response_format.

Tool calls

Jinja chat templates, with parsed and streamed tool-call deltas.

AND ALSO

Careful by design.

Context shift

A slot that fills its context drops the oldest tokens and keeps going.

Multimodal, no fetching

Image and audio as data: URIs only. The engine performs no I/O for a request.

Safetensors to GGUF

A numpy-only converter turns a Hugging Face Llama directory into GGUF on first load.

BY THE NUMBERS

Small on purpose.

1

model per process

9

HTTP endpoints

4

prebuilt platforms

6

build targets: CPU to ROCm

Remote media URLs are refused with 400, by design.

GET STARTED

Two commands to a served model.

terminal

```
$ npm ci && npm run build
$ ./build/onyx-engine -m model.gguf --port 8080
# add -np 4 for four concurrent slots
```

v0.4.0. Prebuilt for linux-x64, darwin-arm64, darwin-x64 and win32-x64.

github.com/JX-Holdings-LLC/Onyx-Engine